Code-Level Disablement

Overview

This document outlines the investigation and technical approach for implementing code-level disablement of suppliers and their related credentials in the NDC system.

Objectives

The goal of this investigation is to establish a mechanism that:

- ✓ **Disables suppliers and credentials directly at the code level** — ensuring system-enforced deactivation
- ✓ **Prevents UI-based modifications** — business users cannot re-enable or modify locked suppliers through the admin interface
- ✓ **Provides permanent deactivation control** — specific suppliers requiring system-enforced deactivation remain protected
- ✓ **Prevents unauthorized reactivation** — accidental or intentional reactivation via the UI is blocked

This approach ensures **tighter control over supplier availability** and enforces business rules at the technical layer.

Technical Approaches

1

Service Layer Enforcement

Concept: Introduce a `SystemLocked` property to the Supplier model. The service layer validates this property and rejects any enable or modify requests when `systemLocked = true`.

Implementation Details

1. Database Schema Update

- Add a `SystemLocked` boolean column to the suppliers table
- Set default value to `false`

2. Service Layer Validation

- In `SupplierApplicationService`, create a validation function to check the `systemLocked` column
- When `systemLocked = true`, skip any state changes including:
  - Approval requests
  - Update operations
  - Status modifications

3. Credential Protection

- In `SupplierCredentialService` class, retrieve the parent supplier for each credential
- Check the `systemLocked` status before allowing credential modifications

2

Database Trigger Enforcement

Concept: Implement database-level triggers that prevent updates to the `enabled` status or credentials when `SystemLocked = true`.

Implementation Details

1. Database Schema Update

- Add a `SystemLocked` boolean column to the suppliers table
- Set default value to `false`

2. Supplier Protection Trigger

- Create a `BEFORE UPDATE` trigger on the `suppliers` table
- Logic:
  - Check if `systemLocked = true`
  - If attempting to modify `enabled` or other protected fields
  - Raise an error and block the operation

3. Credential Protection Trigger

- Implement a trigger on the `credentials` table
- Prevent changes to the `active` status for credentials that reference suppliers with `systemLocked = true`

3

Enum-Based Filtering

Concept: Add an enum containing all locked suppliers and filter them out from the supplier configuration modal.

Implementation Details

1. Create Locked Supplier Enum

- Add a `LockedSupplier` enum with all locked suppliers
- Define each locked supplier as an enum value

2. Configuration Modal Filtering

- Check whether the `supplierConfigModal` has a locked supplier from this enum
- If found, remove it from the `supplierConfigModal` list

4

Code-Level Boolean Flag

Concept: Add a boolean flag directly to the required supplier at the code level to control its lock status.

Implementation Details

1. Add Boolean Flag

- Add a boolean flag to the specific supplier configuration
- Set the flag at the code level for suppliers requiring permanent lock

2. Validation Logic

- Check for this boolean flag before any supplier activation
- If the flag is `true`, lock this supplier
- Prevent users from activating locked suppliers

Comparison Matrix

Aspect	Approach 1: Service Layer	Approach 2: Database Trigger
Enforcement Level	Application layer	Database layer
Flexibility	More flexible, easier to modify logic	More rigid, requires migration to change
Performance	Additional service calls	Native database operation
Error Handling	Controlled application errors	Database-level exceptions
Bypass Risk	Can be bypassed by direct DB access	Cannot be bypassed at all

Conclusion

Both approaches provide robust mechanisms for enforcing supplier disablement at the code level. The choice between them depends on:

- **Security requirements** — Approach 2 offers stronger protection
- **Flexibility needs** — Approach 1 allows easier business logic modifications
- **System architecture** — Consider existing patterns in the codebase

---

A **hybrid approach** combining both service-layer validation and database triggers may provide the optimal balance of flexibility and security.